Mobile Applications

CSE118

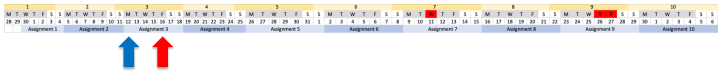
Fall 2026

Android V - Compose & Networking

Baskin Engineering

Notices

- **Assignment 3** due 23:59 **Sunday**
- **Windows** (or Linux or Intel Mac) users must attend class **Friday**
  - Make sure to bring your laptop
  - Confirmation Survey due **Tuesday**



UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Today's Lecture

- Jetpack Compose UIs
- Repositories
- Composed Navigation
- RESTful APIs
- API Authentication
- Making HTTPS calls from Android
- The UI Thread and Background Threads
- Loosely Coupled UIs
- Comosable Testing
- Assignment 3 - Introduction

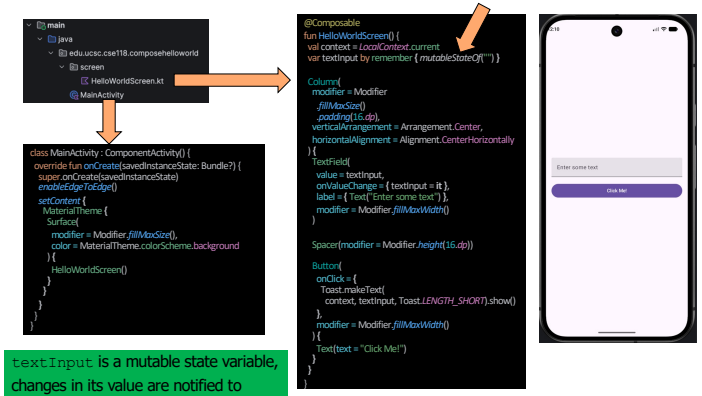
UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Constructing Android UIs

- Traditional: XML Layouts
  - XML layout documents
  - Reference UI elements in them from Kotlin
  - Most existing code works this way
- Current: Jetpack Compose
  - Everything written in Kotlin
  - Introduces important "modern" concepts including state
  - Most new code is written this way

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Composable Hello World



`textInput` is a mutable state variable, changes in its value are notified to interested composables

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Repositories

- Pattern based Kotlin classes that hold (or know how to get) values for instances of a *single* Data Class
- Non-trivial ViewModels use Repositories to supply data to Views within Activities, Fragments and Composables
- Data Classes in the Examples:

```
data class Stadium (
  val name: String,
  val location: String? = null,
  val capacity: Int? = null,
  val built: Int? = null
)
```

Default Values

```
data class BallClub (
  val name: String,
  val code: String,
  val stadium: Stadium
)
```

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

ViewModel / Repository Delegation

```
class BallClubRepo(context: Context) {
  val clubs: List<BallClub> = context.resources.openRawResource(R.raw.ball_clubs).use { stream ->
    InputStreamReader(stream).use { reader ->
      Gson().fromJson(reader, Array<BallClub>::class.java).toList()
    }
  }
}
```

Enhancement? The repository could cache the collection rather than reading it from the raw resource each time

```
class SharedViewModel(application: Application) : AndroidViewModel(application) {
  private val repo = BallClubRepo(application)

  val clubs: List<BallClub>
    get() = repo.clubs

  var selectedClub by mutableStateOf<BallClub?>(null)
    private set

  fun selectClub(club: BallClub) {
    selectedClub = club
  }
}

// Shared by the navigation title, home screen, and detail screen.
val LocalClubViewModel = compositionLocalOf<SharedViewModel> {
  error("Club ViewModel has not been provided")
}
```

The ViewModel asks the repository for the club collection without knowing how it gets them

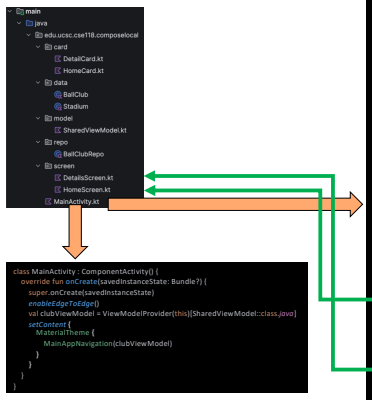
UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Android Navigation

- Traditional
  - XML Navigation Definitions
  - Global Navigation Controller
  - ViewModels for data sharing
  - Observers listen for changes in mutable data (didn't do this in A2)
- Current
  - Composables created in Kotin
  - NavHostController created by Compose
  - ViewModels for data sharing
  - Mutable data stored as state

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Compose Navigation




```
private const val HOME = "home"
private const val DETAILS = "details"

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MainAppNavigation(clubViewModel: SharedViewModel) {
  val navController = rememberNavController()
  val backStackEntry by navController.currentBackStackEntryAsState()
  val isDetails = backStackEntry?.destination?.route == DETAILS
  val title = if (isDetails) clubViewModel.selectedClub?.name.orEmpty() else "MLB Clubs"

  CompositionLocalProvider(LocalClubViewModel provides clubViewModel) {
    Scaffold(
      topBar = {
        TopAppBar(
          title = { Text(title) },
          navigationIcon = {
            if (isDetails) {
              IconButton(onClick = { navController.popBackStack() }) {
                Icon(
                  imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                  contentDescription = "Back"
                )
              }
            }
          }
        )
      }
    ) { innerPadding ->
      NavHost(
        navController = navController,
        startDestination = HOME,
        modifier = Modifier.padding(innerPadding)
      ) {
        composable(HOME) {
          HomeScreen(onNavigateToDetails = {
            navController.navigate(DETAILS) {
              launchSingleTop = true
            }
          })
        }
        composable(DETAILS) {
          DetailsScreen()
        }
      }
    }
  }
}
```

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Making RESTful API calls from Android

- Must give the App permission to talk to the Internet
  - In `app/manifests/AndroidManifest.xml`

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

- Must be done over HTTPS
  - Many 3rd party packages, but we'll use built-in classes

```
import java.net.URL
import javax.net.ssl.HttpsURLConnection
```

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Ball Clubs RESTful API

GET	/ballclub	Returns *summaries* of ball clubs available to the authenticated user
GET	/ballclub/{code}	Returns *fully hydrated* ballclub with the supplied code (e.g., "BOS")

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.


```
class BallClubRepo {
  private val gson = Gson()
  private val baseUrl = "https://cse118.com/api/v2/ballclub"

  suspend fun getClubs(): List<BallClub> = withContext(Dispatchers.IO) {
    gson.fromJson(getJson(baseUrl), Array<BallClub>::class.java).toList()
  }

  suspend fun getClub(code: String): BallClub = withContext(Dispatchers.IO) {
    val encodedCode = URLEncoder.encode(code, "UTF-8")
    gson.fromJson(getJson("$baseUrl/$encodedCode"), BallClub::class.java)
  }

  private fun getJson(url: String): String {
    val connection = URI(url).toURL().openConnection() as HttpURLConnection
    try {
      connection.setRequestProperty("Accept", "application/json")
      val status = connection.responseCode
      if (status != HttpURLConnection.HTTP_OK) {
        throw IOException("Ball club request failed (HTTP $status)")
      }
      return connection.inputStream.bufferedReader().use { it.readText() }
    } finally {
      connection.disconnect()
    }
  }
}
```

Repository for Ball Clubs

URL of the API endpoint

JSON is an array of ball clubs so parse into a Kotlin List of them

Make sure ballclub code is URL safe*
JSON is a single ball club so parse into one

HTTP Method to use is GET (default)

Connection will send plain text (default) and receive JSON

Throw an exception if request was not ok

If request ok, return response body as a string which will be JSON

Tidy up before returning

*Arguably redundant

"suspend" functions have the ability to pause (suspend) their execution and resume later without blocking the underlying thread

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

UI Thread and Background Threads

- Each Activity has a single thread of execution for it's UI
  - Network requests **MUST NOT** be made in this thread
- Long running tasks such as network requests must be done in their own background threads
- On completion, these background threads can notify the UI thread that new data is available
- In Kotlin, background threads are created using a **Coroutine**
- Marking a function "suspend" creates a Coroutine
- Coroutines, typically, announce a change in LiveData (XML dev) or State (Compose Dev)

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.


```
class SharedViewModel(application: Application) : AndroidViewModel(application) {
  private val repo = BallClubRepo()

  var clubs by mutableStateOf<List<BallClub>>(emptyList())
    private set

  var error by mutableStateOf<String?>(null)

  var selectedClub by mutableStateOf<BallClub?>(null)
    private set

  init {
    viewModelScope.launch {
      try {
        clubs = repo.getClubs()
      } catch (e: CancellationException) {
        throw e
      } catch (e: Exception) {
        error = e.message ?: "Unable to load ball clubs"
      }
    }
  }

  fun selectClub(club: BallClub) {
    selectedClub = club
    error = null
    viewModelScope.launch {
      try {
        selectedClub = repo.getClub(club.code)
      } catch (e: CancellationException) {
        throw e
      } catch (e: Exception) {
        error = e.message ?: "Unable to load ball club details"
      }
    }
  }
}
```

When the ViewModel is created, a new thread is created to call a repository "suspend" method to get the clubs from the API without blocking the UI thread

When a ball club is selected, a new thread is created to get the club details from the API, the partially hydrated club being set as the hydrated value temporarily, when the fully hydrated version is returned by the API any UI element interested in the `selectedClub` state will be notified of its new value

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Loosely Coupled UIs

- It's a truly horrendous anti-pattern (bad practice) to have fragments / composables communicate directly
- They should *always* communicate via a shared ViewModel

```
@Composable
fun HomeCard(clubIndex: Int, onClick: () -> Unit, modifier: Modifier = Modifier) {
  val clubViewModel = LocalClubViewModel.current
  val club = clubViewModel.clubs[clubIndex]
  Card(onClick = {
    clubViewModel.selectClub(club)
    onClick()
  }, modifier = modifier) {
    Column(
      modifier = Modifier.padding(16.dp),
      verticalArrangement = Arrangement.spacedBy(4.dp)
    ) {
      Text(
        text = club.name,
        style = MaterialTheme.typography.bodyLarge,
        fontWeight = FontWeight.Bold
      )
      Text(
        text = club.stadium.name,
        style = MaterialTheme.typography.bodyMedium
      )
    }
  }
}
```

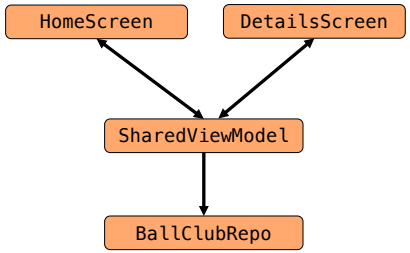
When a ball club is clicked on the home screen, the card displaying it sets club its displaying to be the one selected and calls the `onClick` handler passes to its constructor

The card UI component has ***NO IDEA*** what happens in the app because it did this

Big Win? We can change to entire flow / navigation of the app without having to change individual Composables

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

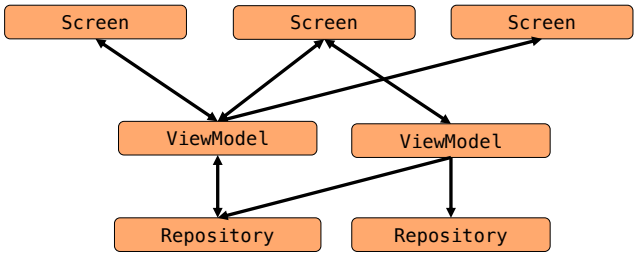
Loose Coupling in the Ball Club App



Fragments / Composables do not communicate with Repositories
Fragments / Composables do not communicate with each other

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Loose Coupling in General



Fragments / Composables do not communicate with Repositories
Fragments / Composables do not communicate with each other

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Composable Testing

- Can't use Espresso / Barista as they assume XML fragments
- Instead, use `androidx.compose.ui.test` package

```
@RunWith(AndroidJUnit4::class)
class ClubNavigationTest {
    @get:Rule
    val compose = createAndroidComposeRule<MainActivity>()

    @Test
    fun homeDisplaysTitle() {
        compose.onNodeWithText("MLB Clubs").assertIsDisplayed()
    }

    @Test
    fun homeDoesNotHaveBack() {
        compose.onNodeWithContentDescription("Back").assertDoesNotExist()
    }

    @Test
    fun homeDisplaysClub() {
        compose.onNodeWithText("Arizona Diamondbacks").assertIsDisplayed()
    }

    @Test
    fun homeDisplaysStadium() {
        compose.onNodeWithText("Chase Field", useUnmergedTree = true).assertIsDisplayed()
    }
}
```

Create an instance of the activity under test

Check expected title is visible

Check back button is not visible

Check known club is visible

Check known club's stadium is visible

Golden Rule: One assertion per test

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Composable Network Testing

- Calls to the API take a finite amount of time and happen in non-UI threads
- Hence, data does not appear in the app instantaneously
- Solution?

```
@Before
fun waitForClubs() {
    compose.waitUntil(timeoutMillis = 15_000) {
        compose.onAllNodes(hasText("Arizona Diamondbacks"))
            .fetchSemanticsNodes().isNotEmpty()
    }
}
```

Before each test, wait up to 15 seconds for a known club to appear before asserting anything

```
private fun selectClub(name: String) {
    compose.onNodeWithText(name).performClick()
    compose.waitUntil(timeoutMillis = 15_000) {
        compose.onAllNodes(hasText("Built:", substring = true) and
            hasText("Built: Unknown").not())
            .fetchSemanticsNodes().isNotEmpty()
    }
}
```

Helper function to select the desired club then wait for up to 15 seconds for its details to appear

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Assignment 3

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

The `cse118.com` API

Endpoints are at: `https://cse118.com/api/v2`

POST	/login	Use your UCSC email address, and student number
GET	/workspace	Returns all workspaces available to the authenticated user
GET	/workspace/{id}/channel	Returns all channels in the workspace with the supplied id
GET	/channel/{id}/message	Returns all messages in the channel with the supplied id
POST	/channel/{id}/message	Creates new message in the channel with the supplied id
DELETE	/message/{id}	Deletes the message with the supplied id

Explore the API @:

`https://cse118.com/api/v2/docs/`

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Assignment 3 - Introduction

- Basic 60% (effectively Assignment 2 but getting data over the network)
  - Log in to API and show a list of Workspaces
  - On selecting a Workspace, show list of Channels
  - On selecting a Channel, show list of Messages
  - On selecting a Message, show its details
- Advanced 15%
  - In a Channel's list of Messages, allow users to create a new Message
- Stretch 15%
  - In a Channel's list of Messages, allow users to delete Messages if they have permission to do so
- Extreme 10%
  - Your tests for Advanced and Stretch work against the model solution

Tests for Basic Provided - Write your own for Advanced and Stretch

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Assignment 3 - Secret Sauce



- When accessing API endpoints, you need to supply the access token (JWT) returned by a successful `/api/v2/login` as an HTTP request authorisation property:

```
setRequestProperty("Authorization", "Bearer ${accessToken}")
```

- Sequence is:
  - Call `/api/v2/login` with your email address and password
  - Save the returned access token
  - Use that access token whenever you call a secure API endpoint

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.

Next Lectures

- Wednesday
  - iOS - Introduction
- Friday
  - Mandatory Workshop for Windows / Linux / Intel Mac users

UCSC BSOE CSE118 Fall 2026. Copyright © 2022-2026 David C. Harrison. All Rights Reserved.